AI for Data Cleaning & Classification

Using LLM APIs with Real Datasets

Groq · Llama 3.1 · Databricks · Delta Lake

60 min | Hands-on | Free Tier | Beginner–Intermediate

ANALYTICSWITHANAND | Anand Kumar Jha

Session agenda

60 minutes · 5 segments · fully hands-on

01	**Setup & Motivation** Groq API setup · why LLMs for data quality	0–10 min
02	**Data Cleaning with LLM** Prompts · few-shot examples · apply to dataset	10–25 min
03	**Zero-Shot Classification** JSON output · single-label · multi-label	25–40 min
04	**Evaluation & Cost Awareness** Gold-standard check · token cost · dedup cache	40–52 min
05	**Production Patterns** Pandas UDF · Delta Lake · architecture	52–60 min

Why LLMs for data quality?

Traditional approach fails on...

- Inconsistent casing: 'APPLE INC', 'apple inc.'
- Typos: 'Gooogle LLC' vs 'Google'
- Format variants: 'USA', 'U.S.A.', 'united states'
- Free-text tickets needing classification

LLMs understand context

- Fix typos using world knowledge
- Standardise formats intelligently
- Classify free-text without labels
- Handle ambiguity naturally

Rule of thumb

Use LLMs only where language understanding is needed. If regex or str.replace() can solve it — don't pay for an API call.

Groq API | Llama 3.1 8B | Databricks | Delta Lake | Free Tier

Tech stack

Everything free · no credit card needed · works in all regions

G

Groq

LLM inference engine

Free API key at console.groq.com
30 RPM · 500K tokens/day

L

Llama 3.1 8B

Open-source LLM

Meta's model running on Groq
Fast · deterministic at temp=0

D

Databricks

Compute platform

Community Edition — free
Spark + Delta Lake included

D

Delta Lake

Storage layer

ACID transactions on data
Write enriched output as Delta table

01 Setup — Groq API in Databricks

3 lines to get started

```
# Install Groq SDK
%pip install groq -q
dbutils.library.restartPython()
```

```
from groq import Groq

GROQ_API_KEY = "gsk_XXXX..."  # console.groq.com

client = Groq(api_key=GROQ_API_KEY)

def call_llm(prompt: str, max_tokens=256) -> str:
    resp = client.chat.completions.create(
        model="llama-3.1-8b-instant",

messages=[{"role":"user","content":prompt}],
        temperature=0,
        max_tokens=max_tokens,
    )
    return
resp.choices[0].message.content.strip()

# Smoke test
```

Key concepts

temperature=0

Deterministic output — same input always gives same result. Critical for data tasks.

call_llm() wrapper

Single function used everywhere. Swap model or provider in one place.

llama-3.1-8b-instant

Fast, free, excellent for data cleaning and classification tasks.

Groq free tier

30 RPM · 6K TPM · 500K tokens/day — enough for all demos today.

02 Data Cleaning with LLM

Precise prompt ⟶ consistent, parseable output

```
def clean_with_llm(value, field_type, examples=""):
    prompt = f"""You are a data cleaning assistant.
Task: Clean this {field_type}.
Rules:
- Return ONLY the cleaned value — no explanation, no
quotes
- Correct spelling and casing
- Use the most official/standard form
{f'Examples:{examples}' if examples else ''}
Input: {value}
Cleaned:"""
    try:
        return call_llm(prompt, max_tokens=64)
    except Exception as e:
        return value  # fail gracefully
```

Before ⟶ After

appl inc	⟶Apple
Gooogle LLC	⟶Google
MSFT Corp	⟶Microsoft
amzn inc	⟶Amazon
usa / U.S.A / u.s.	⟶United States
$1,200.50 / 1200	⟶1200.50 (regex)

Revenue cleaning uses str.replace() not LLM — structured patterns don't need language understanding.

03 Zero-Shot Classification

No labelled training data needed — the model already understands business language

```
CATEGORIES =
["Billing","Technical","Shipping","Returns","Account","Other"
]

def classify_ticket(text):
    prompt = f"""Classify the ticket into ONE of: {',
'.join(CATEGORIES)}
Rules:
- Return ONLY valid JSON — no markdown, no extra text
- confidence: high | medium | low
- reason: one sentence under 15 words

Ticket: {text}
JSON output:"""
    try:
        raw = call_llm(prompt, max_tokens=128)
        raw = re.sub(r"```(?:json)?|```", "", raw).strip()
        result = json.loads(raw)
        assert "category" in result
        return result
    except Exception:
        return {"category": "Other", "confidence": "low"}
```

3 golden rules

- Say 'Return ONLY valid JSON'
- Strip markdown fences (```json)
- Always try/except on json.loads()

Example output

```
{
  "category": "Billing",
  "confidence": "high",
  "reason": "User reports
incorrect charge amount"
}
```

Single-label vs multi-label classification

One prompt change unlocks multi-label output

Single-label

One category per ticket

```
...into exactly ONE of:
Billing, Technical...

Returns:
{
  "category": "Billing",
  "confidence": "high"
}
```

Change ONE word →

Multi-label

One or more categories per ticket

```
...ONE OR MORE of:
Billing, Technical...

Returns:
{
  "categories": ["Billing","Account"],
  "primary": "Billing",
  "confidence": "high"
}
```

Use case: 'charged twice and now account is locked' → both Billing and Account

04 Evaluation & Cost Awareness

Measure quality · track tokens · cache duplicates

Accuracy check

87.5%

7 / 8 gold-standard tickets correct

Spot-check even 50 rows before shipping

Token cost per call

Model	Per 1M tokens
Groq free	$0.00
Llama 3.1 8B paid	$0.05
Gemini 1.5 Flash	$0.075
GPT-4o	$5.00

Dedup cache

75%

API calls saved on 20 rows with 5 unique company names

```
cache = {}
def cached_clean(val):
    if val not in cache:
        cache[val] = clean_with_llm(val, "company name")
    return cache[val]

# 20 rows, 5 unique → only 5 API calls made
```

05 Production Patterns

Scale with Spark · write to Delta Lake · schedule as a job

Pandas UDF — for > 10K rows

```
@pandas_udf(StringType())
def classify_udf(series: pd.Series) -> pd.Series:
    client = Groq(api_key=KEY_BROADCAST.value)
    # runs on each Spark partition in parallel
    ...
```

Write to Delta Lake

```
(spark.createDataFrame(final_df)
  .write.format("delta")
  .mode("overwrite")
  .saveAsTable("cleaned_tickets_groq"))
```

Hybrid production architecture

Dedup cache on LLM layer · confidence filtering before Delta write · schedule with Databricks Workflows

When to use LLM vs rule-based

LLMs add cost and latency — use them only where they add value

Task	Approach	LLM?
Fix 'Gooogle' → 'Google'	LLM — needs world knowledge	Yes
Classify free-text tickets	LLM zero-shot	Yes
Extract SKU from notes	LLM NER prompt	Yes
Standardise 'USA'/'U.S.A.'	Mapping dict / replace()	No
Parse dates '01-Jan-24'	dateutil.parser / regex	No
Remove $ and commas	str.replace() / regex	No
Detect missing zip code	regex / len() check	No

Key takeaways

Precise prompts

Return ONLY the cleaned value — no explanation, no quotes. This is the single most important rule.

JSON output

Force JSON, strip markdown fences, always try/except. Never trust raw LLM text.

call_llm() wrapper

One function wraps everything. Swap model or provider in one place — nothing else changes.

Dedup cache

Cache by (normalised_value, field_type). Typically saves 60–80% of API calls in real pipelines.

LLM vs rule-based

Regex for structured patterns. LLM only where language understanding is actually needed.

Pandas UDF

Broadcast API key, run classify on each partition. Scales to millions of rows on Databricks.

Next steps & resources

Continue building after this session

1 **Try with your data**

Load your own CRM / ERP table and run the full pipeline

2 **Level up the model**

Switch to llama-3.3-70b-versatile for harder classification tasks

3 **Add MLflow logging**

Track accuracy, token usage, and model version over time

4 **Schedule the pipeline**

Wrap the notebook as a Databricks Workflow — run nightly

5 **Explore Groq models**

mixtral-8x7b-32768 for long-context · gemma2-9b-it as alternative

Resources

Groq free key
console.groq.com

Groq docs
console.groq.com/docs

Llama models
groq.com/models

Databricks CE
community.cloud.databricks.com

Session notebook
ANALYTICSWITHANAND